



ADVENTIST UNIVERSITY OF CENTRAL AFRICA
MSc in Big Data Analytics

Distributed Multi-Model Analytics for E-Commerce Data

Final Project Technical Report

Submitted by:
SHEMA Joshua RWAKA
101342
June 2026

1. Introduction

This report documents the design, implementation, and findings of a Big Data Analytics project built around a simulated e-commerce platform. The goal was to harness the complementary strengths of three distinct technologies: MongoDB for flexible document storage, HBase for time-series and sparse data, and Apache Spark for large-scale distributed processing.

The dataset was synthetically generated using a Python script and models the kind of data a real e-commerce company would collect over a 90-day period. It includes 10,000 users, 5,000 products across 25 categories, 500,000 transactions, and 2,000,000 browsing sessions. Working with data at this scale meant that every design decision had to be deliberate. You cannot simply dump everything into one database and hope for the best.

Each tool had to earn its place in the architecture.

What made this project genuinely interesting was not just loading data into databases but thinking carefully about why one database fits better than another for a specific use case.

The Customer Lifetime Value analysis in particular brought this to life. Combining user demographics from MongoDB with transaction histories through Spark made it possible to answer business questions that no single database could answer alone.

2. System Architecture Overview

The system is built around three core layers, each handling a different aspect of the data pipeline.

The storage layer uses MongoDB as the primary document store for product catalogs, user profiles, and transaction records. These are naturally document-shaped. A transaction, for example, contains embedded line items, a payment method, and a reference back to the session that generated it. MongoDB handles this nesting elegantly without needing joins. HBase was designed to sit alongside MongoDB as the wide-column store for high-volume, time-ordered session and browsing data, where row keys combine a user ID with a timestamp to enable fast retrieval of any user's recent activity.

The processing layer is Apache Spark, which reads JSON files directly and processes them as distributed DataFrames. Spark does the heavy lifting: cleaning and normalizing raw data, running cohort analyses, computing product affinity scores, and estimating customer lifetime value by joining datasets that would be expensive to query live in a transactional database.

The analytics and visualization layer sits on top, using Python's matplotlib and seaborn libraries to turn query results into meaningful charts.

Component	Role	Data Handled
MongoDB	Document store	Products, Users, Transactions, Sessions (subset)
HBase (Simulated)	Wide-column store	Time-series session data, product metrics
Apache Spark	Distributed processing	All JSON files, batch analytics, SQL queries
Python / Matplotlib	Visualization	Charts from query results

3. Data Modeling Decisions

3.1 MongoDB Schema Design

The MongoDB collections were designed around the query patterns the project required rather than around normalization rules.

For the products collection, each document contains the full price history as an embedded array rather than in a separate table. This means that retrieving a product and seeing its complete pricing evolution requires just one database read. The same logic applies to transactions: each transaction document embeds its line items directly, so calculating order totals or analyzing what was purchased together does not require any joins.

User profiles are kept lean in MongoDB. The `geo_data` field nests city, state, and country together since they are almost always queried together. The `registration_date` and `last_active` fields are stored as ISO strings, which Spark can parse efficiently when doing cohort analysis.

Sessions were partially loaded into MongoDB (two out of twenty files, representing 200,000 sessions). While sessions could in principle live entirely in MongoDB, the volume makes HBase a more appropriate home for them at scale because HBase handles wide, sparse rows with billions of entries more gracefully than MongoDB handles very large collections of similarly structured documents.

3.2 HBase Schema Design

HBase organizes data very differently from MongoDB. There are no documents or collections. Instead, everything is a row identified by a row key, and the design of that row key determines how fast reads and writes will be.

For the `user_sessions` table, the row key was designed as `user_id + timestamp`. This means all sessions for the same user are physically stored together on disk, sorted by time. Retrieving the last thirty days of activity for a specific user becomes a simple range scan rather than a full table scans. A reverse timestamp variant (where timestamps are subtracted from a large constant) would put the most recent sessions first, which is often what you want when building a user activity feed.

For the `product_performance` table, the row key is `product_id + date`. This allows efficient retrieval of how a specific product performed on any given day, as well as time-range scans to see weekly or monthly trends.

Table	Row Key Design	Column Families	Query Pattern
<code>user_sessions</code>	<code>user_id + timestamp</code>	<code>session_info</code> , <code>device</code> , <code>geo</code> , <code>behavior</code>	All sessions for user X in last N days
<code>product_performance</code>	<code>product_id + date</code>	<code>metrics</code> , <code>inventory</code>	Product X performance over 90 days

3.3 Why MongoDB vs HBase

The decision about what goes where comes down to data shape and access patterns. MongoDB excels when documents have varying structures, when related data should be embedded together, and when you need rich query capabilities including aggregation pipelines, text search, and flexible filtering. Products, transactions, and user profiles all benefit from this.

HBase excels when you have billions of rows that are accessed by a known key, when the data is time-ordered, and when writes need to be extremely fast. Session data fits this profile perfectly. A single user can generate dozens of sessions per day, and most queries against session data follow the pattern 'give me all activity for user X between time A and time B'.

4. Apache Spark Processing Pipeline

4.1 Batch Processing

Spark was used to process the full 500,000-transaction dataset as well as the user and product files. The processing pipeline followed three stages.

First, the raw JSON files were loaded into Spark DataFrames. Spark's JSON reader handled the nested structures automatically, inferring the schema from the data. This meant that embedded arrays like the items array in transactions were accessible as ArrayType columns.

Second, data quality checks were run. The null check confirmed that no transactions were missing a total amount or a user ID, which means the dataset is clean and ready for analysis without imputation or filtering.

Third, the cleaned DataFrames were registered as SQL temporary views, which allowed Spark SQL queries to express complex analytics in familiar SQL syntax while still benefiting from Spark's distributed execution engine under the hood.

4.2 Spark SQL Analytics

Three main Spark SQL queries were implemented. The daily revenue query grouped transactions by the date portion of the timestamp and summed totals, revealing that June 4, 2026 was the highest-revenue day at \$5.43 million from 5,708 transactions. The payment method analysis showed that credit card and PayPal together accounted for roughly half of all transactions by volume, while bank transfers had the highest average order value at \$1,104.

The cohort analysis joined the transactions table with the users table on user_id, then grouped by the year-month of each user's registration date. This showed that users who registered in October 2025 generated the most total revenue (\$82 million), which makes sense since they had the most time to accumulate purchases within the 90-day observation window.

Analysis	Key Finding
Daily Revenue	Peak day: June 4, 2026 at \$5.43M from 5,708 transactions
Payment Methods	Credit card leads at \$117M; bank transfer has highest avg order (\$1,104)
Cohort Analysis	October 2025 registrants generated most revenue (\$82M total)
Data Quality	Zero null values across 500,000 transactions

5. Integrated Analytics Workflow

5.1 Customer Lifetime Value (CLV) Estimation

The most interesting analysis in this project was the Customer Lifetime Value estimation. The business question was straightforward: which customers are genuinely most valuable, and what separates them from average customers?

The analysis used Spark to join transaction data with user profiles, then computed four metrics per user: total orders placed, total amount spent, average order value, and the date range of their purchase history. The CLV score was defined as total spend divided by number of orders, giving a revenue-per-order figure that reflects how much a customer spends each time they engage.

The top customer, user_005142, spent \$80,881 across 73 orders over the 90-day period, with an average order value of \$1,108. Interestingly, user_004477 placed fewer orders (58) but had a much higher average order value (\$1,308), suggesting a different buying pattern: fewer but larger purchases. Both types of customer are valuable but would respond differently to marketing. The high-frequency buyer might respond to loyalty rewards, while the high-value buyer might be more influenced by premium product launches.

This analysis could not have been done efficiently in MongoDB alone. The join between two large collections, aggregation across hundreds of thousands of rows, and sorting all happen in Spark's distributed memory, which is orders of magnitude faster than doing the equivalent in MongoDB's aggregation pipeline for this volume.

5.2 Product Affinity Analysis

The product affinity analysis attempted to find pairs of products that are frequently purchased together. The approach used a self-join on the transactions table: for every transaction containing more than one item, the first and second product IDs were extracted and paired.

The results showed that many product pairs were bought together three times each. This is expected behavior in a synthetic dataset where purchase combinations are essentially random. In a real e-commerce system with genuine customer preferences, you would expect much stronger affinities to emerge. The analysis framework is correct and would surface meaningful patterns given real behavioral data.

5.3 Funnel Conversion Analysis

The HBase simulation revealed conversion rates by device type using 1,000 sampled sessions. Mobile users converted at 21.6%, tablet users at 19.5%, and desktop users at 18.3%. This finding suggests that mobile users, despite having smaller screens and potentially more friction in the checkout process, are actually more likely to complete a purchase once they start browsing. This could reflect the fact that mobile users in this dataset were more intentional about visiting the site.

6. Visualizations and Key Insights

Four visualizations were produced to communicate the analytical findings.

The revenue by payment method chart showed a bar chart comparing six payment methods. Credit card and PayPal are nearly tied at around \$117 million each, while bank transfer and gift card each sit at around \$67 million. Crypto and Apple Pay trail behind at roughly \$49 million each. This distribution matters for platform design: if the platform wanted to optimize revenue, it should ensure the credit card and PayPal checkout flows are as frictionless as possible since they drive the most volume.

The top ten best-selling products chart presented a horizontal bar chart showing quantity sold. The top product, prod_00283, sold 661 units. The spread across the top ten is relatively tight (624 to 661 units), which in a real system would suggest healthy catalog diversity rather than over-reliance on a single SKU.

The conversion rate by device type chart highlighted mobile as the highest-converting device at 21.6%, followed by tablet at 19.5% and desktop at 18.3%. The differences are modest but directionally consistent.

The daily revenue trend chart showed 90 days of revenue plotted as a line with a filled area beneath it. Revenue was relatively stable across the period, hovering between \$4.5 million and \$5.5 million per day, with no dramatic spikes or drops. This stability is characteristic of synthetic data but provides a useful baseline for what steady-state e-commerce performance might look like.

7. Scalability Considerations

The current implementation runs on a single Windows machine, which imposes real limits. The Spark jobs process 500,000 transactions and 2 million sessions sequentially in a single JVM. In a production environment, Spark would run on a cluster of machines, and the same jobs would distribute their work across dozens or hundreds of nodes simultaneously.

MongoDB scales horizontally through sharding. The transactions collection at 500,000 documents is manageable on a single node, but at 500 million documents, you would partition the collection across multiple shards, typically using user_id or a hashed key as the shard key to distribute load evenly.

HBase was designed from the ground up for horizontal scale. Adding more region servers distributes the row key space across more machines. The row key design choices made here (user_id + timestamp, product_id + date) become even more important at scale because they determine which region server handles which rows. A poorly chosen row key creates hot spots where one server handles all the traffic while others sit idle.

The session data at 2 million records is already large enough that loading it all into MongoDB was impractical for this project. At 200 million sessions, HBase would be the right choice without question.

8. Technology Selection Justification

Requirement	Technology Chosen	Why
Store product catalog with price history	MongoDB	Embedded arrays fit naturally; rich query support
Store 500K transactions with line items	MongoDB	Document model handles embedded items; aggregation pipelines for analytics
Time-series user session data at scale	HBase	Row key design enables fast user activity retrieval; designed for billions of rows
Distributed batch processing	Apache Spark	Processes full dataset in memory across partitions; SQL

Complex multi-dataset joins (CLV, Cohort)	Apache Spark	interface is expressive Joins two large tables efficiently; DataFrame API handles nested JSON
Visualizations	Matplotlib / Seaborn	Lightweight, flexible, no server required

9. Limitations and Future Work

9.1 Limitations

The most significant practical limitation encountered during this project was the hardware constraint around Docker and virtualization. HBase was intended to run as a Docker container, which would have allowed real HBase shell commands and actual Thrift API calls through HappyBase. Virtualization was disabled at the BIOS level on the development machine, making Docker Desktop non-functional. The HBase implementation was therefore simulated in Python using dictionaries that mirror HBase's row key and column family structure. The schema design and query logic are architecturally correct, but the queries run against in-memory Python data structures rather than a live HBase cluster.

A second limitation is that the synthetic dataset, while large, does not exhibit real behavioral patterns. Product affinities in the real world emerge from genuine preferences. In synthetic data, purchase combinations are essentially random, which is why the product affinity analysis found only weak signals (pairs bought together just 3 times).

The dataset size also created performance challenges. The full 4GB dataset took considerable time to generate, and loading all 2 million sessions into MongoDB was not practical given available memory and disk speed. Only 200,000 sessions were loaded into MongoDB for the aggregation queries.

9.2 Future Work

If this system were to be extended, several improvements would be worth pursuing. First, running the full stack on a cloud environment such as Google Cloud or Amazon EMR would remove the hardware constraints entirely and allow HBase to run properly. Second, the product affinity analysis would benefit from a proper market basket analysis using the MLlib FPGrowth algorithm, which Spark includes natively. Third, a real-time stream processing layer using Spark Streaming or Apache Kafka could process incoming sessions as they happen rather than in batch, enabling live dashboards and real-time fraud detection.

10. Conclusion

This project demonstrated that no single database technology is the right choice for all aspects of a large-scale e-commerce analytics system. MongoDB proved its value for storing and querying richly structured documents like transactions and product catalogs. HBase showed how its row key design enables efficient time-series retrieval at a scale that would strain a document store. Apache Spark tied everything together by providing the distributed processing power to run analytics across datasets that would be impractical to query live.

The Customer Lifetime Value analysis was the highlight of the project. Seeing how different customers behave, some placing many moderate orders and others making fewer but larger purchases, illustrated how multi-model analytics can surface insights that inform real business decisions. A marketing team armed with CLV scores and device-specific conversion rates can make meaningfully better decisions about where to invest resources.

The challenges encountered, particularly around Java version compatibility and the Docker virtualization constraint, were frustrating but educational. They reinforced that big data systems are not just about writing clever queries. Infrastructure setup, dependency management, and environment configuration are real parts of the job.

References

Apache Software Foundation. (2024). Apache Spark Documentation. <https://spark.apache.org/docs/latest/>

MongoDB, Inc. (2024). MongoDB Manual. <https://www.mongodb.com/docs/manual/>

Apache Software Foundation. (2024). Apache HBase Reference Guide. <https://hbase.apache.org/book.html>

Zaharia, M., et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56-65.

George, L. (2011). *HBase: The Definitive Guide*. O'Reilly Media.